

第 4 章

デバイスの制御と状態同期

クラウドからデバイスへ働きかける手段を、用途とオフライン時の挙動で使い分ける

Direct Method

Device Twin

C2D メッセージ

成功の確認

クラウドからデバイスへ働きかける3つの目的

クラウドからデバイスへ働きかける操作は、目的とオフライン時の要件で使い分ける。

今すぐ操作したい

ファンを今止め、結果を
その場で確認する

● Direct Method

設定を変えたい

送信間隔を10秒に変え、
その状態を保ちたい

● Device Twin

通知・依頼を届けたい

メンテナンス予定や
診断ログ依頼を届ける

● C2D メッセージ

「今やって結果を教えて」は Direct Method、「この状態にしておいて」は Twin、「この通知・依頼を届けて」は C2D。

[1] <https://learn.microsoft.com/ja-jp/azure/iot-hub/iot-hub-devguide-c2d-guidance>

Direct Method・Device Twin・C2D の使い分け

同じ「クラウド→デバイス」でも、要求の形とオフライン時の扱いが異なる。

方法	主な用途	オフライン時の扱い
Direct Method	即時操作し、その場で応答を確認する	接続待ち内に接続できなければ失敗。後日配送用のキューは持たない
Device Twin (Desired)	最終的に期待する構成へ合わせる	期待値が保持され、再接続時に適用できる
C2D メッセージ	デバイスへの一方向の通知・依頼	一定期間キューに保持（既定 TTL 1時間・最大2日）

継続的に維持する構成は Twin に期待値として保持し、個別の通知・依頼は C2D で届ける。すべてを C2D にはしない。

[1] <https://learn.microsoft.com/ja-jp/azure/iot-hub/iot-hub-devguide-c2d-guidance#comparison-of-cloud-to-device-communication-options> [2] <https://learn.microsoft.com/ja-jp/azure/iot-hub/iot-hub-devguide-messages-c2d>

C2D でも制御できるが、Direct Method と何が違うか

C2D でも開始・停止は実装できる。違いは要求・応答と未接続時の扱いにある。

C2D で `{"action": "start"}` を送り、受信ハンドラで送信を開始できる

比較	Direct Method	C2D メッセージ
基本の形	要求と応答	一方向のメッセージ配送
実行結果の確認	応答でステータス・本文を返せる	返送と要求の対応付けをアプリで実装
未接続時の扱い	長期キューには保持しない	有効期限内はキューに保持できる
選ぶときの意図	「今、開始して。結果を教えて」	「これをお届け。受信したら処理して」

遅れて届いた操作を実行してよいかも選定基準。停止中に届いた「開始」を再接続後に実行すると意図と異なる場合がある。

[1] <https://learn.microsoft.com/ja-jp/azure/iot-hub/iot-hub-devguide-c2d-guidance#comparison-of-cloud-to-device-communication-options> [2] <https://learn.microsoft.com/ja-jp/azure/iot-hub/iot-hub-devguide-messages-c2d>

送信成功・応答・実行成功は同じではない

クラウドへの送信成功、デバイスからの応答、アプリの実行完了は別の事実。

観点	Direct Method	C2D メッセージ
クラウドが確認するもの	要求に対するデバイスの応答	IoT Hub への送信・キューへの受け付け
未接続時	接続待ち内に接続できなければ失敗	キューに保持し、有効期限内の受信を待てる
結果の解釈	応答タイムアウトでも未実行とは限らない	送信成功でも受信・処理成功とは限らない

クラウドへ送信成功



デバイスで受信



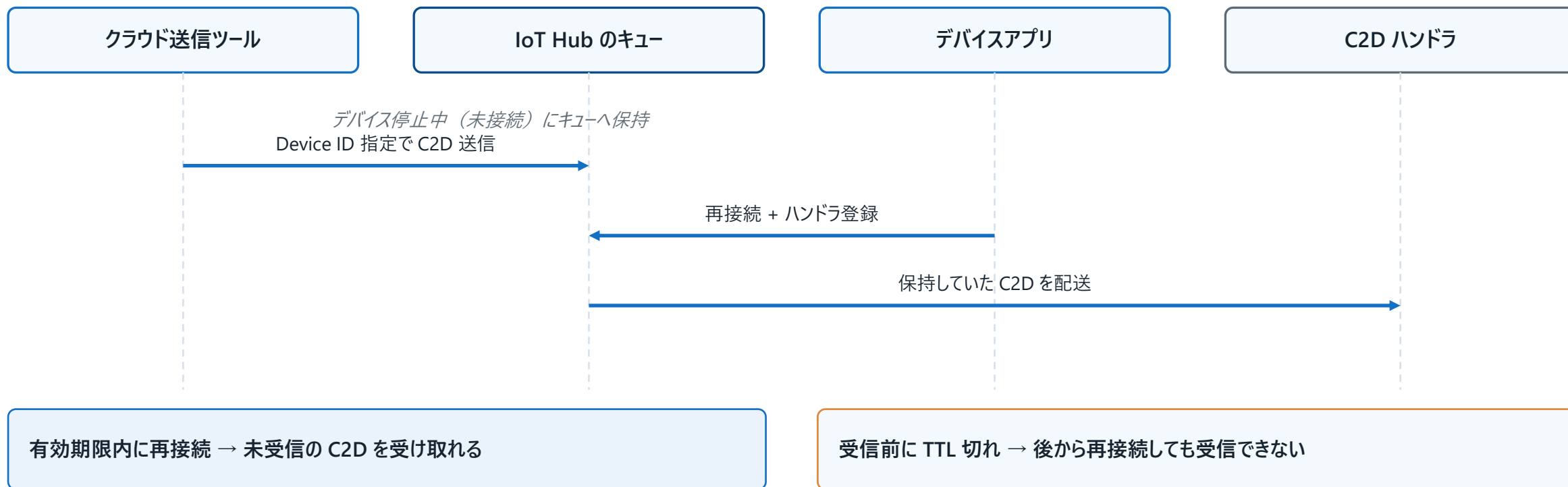
アプリの処理完了（別に確認）

応答タイムアウトは「期限内に結果を確認できなかった」、C2D の送信成功は「IoT Hub が受け付けた」までを意味する。

[1] <https://learn.microsoft.com/ja-jp/azure/iot-hub/iot-hub-devguide-direct-methods#invoke-a-direct-method-from-a-back-end-app> [2] <https://learn.microsoft.com/ja-jp/azure/iot-hub/iot-hub-devguide-messages-c2d>

C2D の保持と再接続後の受信

C2D は IoT Hub のデバイス別キューに保持され、有効期限内に再接続すれば受信できる。



既定 TTL 1時間・最大2日。保持するのは停止中のアプリではなく IoT Hub。

[1] <https://learn.microsoft.com/ja-jp/azure/iot-hub/iot-hub-devguide-messages-c2d> [2] <https://learn.microsoft.com/ja-jp/azure/iot-hub/how-to-cloud-to-device-messaging?pivots=programming-language-python#create-a-device-application>

Device Twin の構造（Desired・Reported・Tags）

Device Twin は、期待する構成・報告された状態・分類情報を保持する。



Desired は期待状態、Reported は実状態、Tags はクラウド側の分類情報。Tags はデバイスへ配送しない。

[1] <https://learn.microsoft.com/ja-jp/azure/iot-hub/iot-hub-devguide-device-twins>

Desired と Reported の状態同期

Desired の更新は適用完了を意味しない。実際の適用は Reported で確認する。



Desired と Reported は非同期。更新した事実と、各デバイスでの適用成功は分けて確認する。

[1] <https://learn.microsoft.com/ja-jp/azure/iot-hub/iot-hub-devguide-device-twins#back-end-operations>

登録した3手段を事例で選び分ける

次の場面で、どの手段を選ぶか説明してください。

- 1 ファンを今すぐ停止し、結果を確認したい。
- 2 オフライン中のデバイスにも、送信間隔の変更を届けて保ちたい。
- 3 「再接続後に診断ログを送ってほしい」と「送信間隔を10秒に保ってほしい」を選び分ける。
- 4 応答タイムアウトと C2D の送信成功から、それぞれ何が分かり、何が分からないか。

次章: IoT Hub が受け取った D2C を、条件に応じて後続サービスへ配送するメッセージルーティングを扱う。